Projekt I przetwarzanie równoległe 2025/26 SS, OpenMP, Vtune

Emilia Bonk 160108
Szymon Urban 160271

Wstęp.

- **Nazwa przedmiotu:** Laboratorium z przetwarzania równoległego
- **Autorzy:** Emilia Bonk (160108), Szymon Urban (160271)
- **Grupa dziekańska:** L09
- **Termin zajęć laboratoryjnych:** Wtorek 9:45-11:15
- **Termin oddania sprawozdania:** 5.05.2026
- **Wersja sprawozdania:** pierwsza
- **Opis zadania:** Celem ćwiczenia była analiza implementacji algorytmu Sita Eratostenesa w środowisku równoległym i sekwencyjnym z wykorzystaniem biblioteki OpenMP. Zadanie polegało na analizie i modyfikacji podanych kodów, z róznymi implementacjami algorytmu, a następnie porównaniu ich wydajności.
- **Adres e-mail:** emilia.bonk@student.put.poznan.pl, szymon.urban@student.put.poznan.pl

1 Informacje techniczne.

Parametr	**Wartość**
Model procesora	17th Gen Intel(R) Core(TM) i7-12700F
Liczba procesorów fizycznych	1
Liczba rdzeni	12
Liczba procesorów logicznych	20
Pamięć cache L1	1,0 MB
Pamięć cache L2	12,0 MB
Pamięć cache L3	25,0 MB
System operacyjny	Windows 10
Kompilator	Intel(R) oneAPI DPC++/C++ Compiler 2025.3.3
Wersja kompilatora	2025.3.3.20260319
Program testujący	Intel VTune Profiler 2026.0.0

Tablica 1: Informacje techniczne dotyczące środowiska testowego

2 Prezentacja i analiza kodów.

K1 - Algorytm sekwencyjny oparty na próbnych dzieleniach.

Oznaczenie skrótowe: SEQ-DIV-BASIC

Algorytm sekwencyjny oparty na metodzie dzielenia próbnego. Najpierw wyznacza liczby pierwsze w zakresie do $\sqrt{n}$, a następnie szeregowo sprawdza podzielność każdej liczby $i \in [m, n]$ przez te dzielniki. Koszt obliczeń jest zmienny – najniższy dla liczb parzystych, najwyższy dla liczb pierwszych.

```
int calculate(int n, int m){
        bool* result = (bool*)malloc((n - m + 1) * sizeof(bool));
        memset(result, true, (n - m + 1) * sizeof(bool));
        int prime_counter = 0;

        bool* primeArray = (bool*)malloc((sqrt(n) + 1) * sizeof(bool));
        memset(primeArray, true, (sqrt(n) + 1) * sizeof(bool));

        if (m <= 0 && n >= 0) result[0 - m] = false;
        if (m <= 1 && n >= 1) result[1 - m] = false;

        for (int i = 2; i * i <= n; i++) {
                for (int j = 2; j * j <= i; j++) {
                        if (primeArray[j] == true && i % j == 0) {
                                primeArray[i] = false;
                                break;
                        }
                }
        }
        for (int i = m; i <= n; i++){
                for (int j = 2; j * j <= i; j++){
                        if (primeArray[j] == true && i % j == 0) {
                                result[i - m] = false;
                                break;
                        }
                }
                if(result[i-m] == true){
                        prime_counter++;
                }
        }
        free(result);
        free(primeArray);

        return prime_counter;
}
```

Listing 1: Kod programu k1.cpp

K2 - Liczby pierwsze wyznaczane równolegle przez dzielenie w zakresie <m,n>.

Oznaczenie skrótowe: PAR-DIV-GUIDED

Wariant równoległy wykorzystujący dzielenie próbne z dyrektywą `schedule(guided)`. Zastosowanie malejących paczek zadań pozwala na efektywne zrównoważenie obciążenia procesora, niwelując różnice w czasie sprawdzania małych i dużych liczb. Brak wyścigów przy zapisie do tablicy wyników.

```
int calculate(int n, int m){
        int prime_counter = 0;

        //...liczenie sita (PrimeArray) jak w kodzie k1
        #pragma omp parallel
        {
        #pragma omp for schedule(guided) reduction(+:prime_counter)
        for (int i = m; i <= n; i++){
                for (int j = 2; j * j <= i; j++){
                        if (primeArray[j] == true && i % j == 0) {
                                result[i - m] = false;
                                break;
                        }
                }
                if(result[i-m] == true){
                        prime_counter++;
                }
        }
        }
        free(result);
        free(primeArray);

        return prime_counter;
}
```

Listing 2: Kod programu k2.cpp

K3 - Sito sekwencyjne bez lokalności dostępu do danych.

Oznaczenie skrótowe: SEQ-SIEVE-NL

Sito Eratostenesa realizowane sekwencyjnie. Zamiast dzielenia, algorytm wykreśla wielokrotności liczb pierwszych $i \leq \sqrt{n}$ w całym zakresie $[m, n]$. Charakteryzuje się niską złożonością obliczeniową $O(n \log \log n)$, ale ograniczoną lokalnością danych przy skokowym dostępie do tablicy wyników.

```
int calculate(int n, int m){
        //...liczenie sita jak we wczesniejszych kodach

        for (int i = 2; i*i <= n; i++){
                if (primeArray[i]) {
                        int firstMultiple = (m / i);
                        if (firstMultiple <= 1) {
                                firstMultiple = i + i;
                        } else if (m % i) {
                                firstMultiple = (firstMultiple * i) + i;
                        } else {
                                firstMultiple = (firstMultiple * i);
                        }
                        for (int j = firstMultiple; j <= n; j+=i) {
                                result[j-m] = false;
                        }
                }
        }
        if (m <= 0 && n >= 0) result[0 - m] = false;
        if (m <= 1 && n >= 1) result[1 - m] = false;

        for (int i = m; i <= n; i++) {
        if (result[i - m]) prime_counter++;
        }

        free(result);
        free(primeArray);

        return prime_counter;
}
```

Listing 3: Kod programu k3.cpp

K3a - Sito sekwencyjne z lokalnością dostępu do danych.

Oznaczenie skrótowe: SEQ-SIEVE-LL

Sekwencyjne sito segmentowe zoptymalizowane pod kątem lokalności danych. Zakres $[m, n]$ dzielony jest na bloki mieszczące się w pamięci cache L1/L2. Przetwarzanie blokowe znacząco przyspiesza operacje zapisu do tablicy wyników poprzez redukcję pudeł cache.

```
int calculate(int n, int m){
    //...alokacja pamieci i liczenie PrimeArray tak samo jak w kodzie k3

    int blockSize = 65536;
    int numberOfBlocks = (n - m) / blockSize;
    if ((n - m) % blockSize != 0) {
        numberOfBlocks++;
    }

    for (int i = 0; i < numberOfBlocks; i++) {
        int low = m + i * blockSize;
        int high = low + blockSize - 1;
        if (high > n) {
            high = n;
        }

        for (int j = 2; j * j <= high; j++) {
            if (primeArray[j] == true) {
                int firstMultiple = (low / j);
                if (firstMultiple <= 1) {
                    firstMultiple = j + j;
                } else if (low % j) {
                    firstMultiple = (firstMultiple * j) + j;
                }  else {
                    firstMultiple = (firstMultiple * j);
                }
                for (int k = firstMultiple; k <= high; k += j) {
                    result[k - m] = false;
                }
            }
        }
    }

        //...zliczanie prime_count i zwalnianie pamieci jak w kodzie k3
}
```

Listing 4: Kod programu k3a.cpp

K4 - Sito równoległe funkcyjne bez lokalności dostępu do danych.

Oznaczenie skrótowe: PAR-SIEVE-NL

Równoległe sito z podziałem funkcyjnym (`schedule(dynamic, 1)`). Wątki pobierają kolejne liczby pierwsze i wykreślają ich wielokrotności w całym zakresie wyników. Występuje zjawisko `false sharing` oraz bezpieczne wyścigi przy zapisie tej samej wartości logicznej pod ten sam indeks.

```
int calculate(int n, int m){
        //...alokacja pamieci i lizcenie PrimeArray jak wczesniej

        int limit = sqrt(n);
        #pragma omp parallel for schedule(dynamic, 1)
        for (int i = 2; i <= limit; i++){
                if (primeArray[i]) {
                        int firstMultiple = (m / i);
                        if (firstMultiple <= 1) {
                                firstMultiple = i + i;
                        } else if (m % i){
                                firstMultiple = (firstMultiple * i) + i;
                        } else {
                                firstMultiple = (firstMultiple * i);
                        }
                        for (int j = firstMultiple; j <= n; j+=i) {
                                result[j-m] = false;
                        }
                }
        }

        //zliczanie prime_count i zwalnianie pamieci jak wczesniej
}
```

Listing 5: Kod programu k4.cpp

K4a - Sito równoległe funkcyjne bez lokalności dostępu do danych z usunięciem redundancji zapisów.

Oznaczenie skrótowe: PAR-SIEVE-NL-A

Modyfikacja algorytmu K4 wprowadzająca warunkowy zapis do tablicy wyników (`if (result[j-m])`). Ma to na celu ograniczenie liczby operacji zapisu i związanych z nimi konfliktów w pamięci cache, co poprawia wydajność przy zachowaniu podziału funkcyjnego.

```
int calculate(int n, int m){
        //...alokacja pamieci i zliczanie PrimeArray jak wczesniej

        int limit = sqrt(n);
        #pragma omp parallel for schedule(dynamic, 1)
        for (int i = 2; i <= limit; i++){
                if (primeArray[i]) {
                        int firstMultiple = (m / i);
                        if (firstMultiple <= 1) {
                                firstMultiple = i + i;
                        } else if (m % i){
                                firstMultiple = (firstMultiple * i) + i;
                        } else {
                                firstMultiple = (firstMultiple * i);
                        }
                        for (int j = firstMultiple; j <= n; j+=i) {
                                if (result[j-m]) result[j-m] = false;
                        }
                }
        }

        //zliczanie prime_counter i zwalnianie pamieci jak w k4
}
```

Listing 6: Kod programu k4a.cpp

K5 - Sito równoległe domenowe z lokalnością dostępu do danych

Oznaczenie skrótowe: PAR-SIEVE-DOM-LL

Równoległe sito segmentowe z podziałem domenowym. Każdy wątek przetwarza niezależne bloki danych mieszczące się w cache, co eliminuje `false sharing` i wyścigi. Zastosowanie `schedule(static)` minimalizuje narzut synchronizacji przy wyrównanym obciążeniu bloków.

```
int calculate(m, n){
        //...alokacja pamieci i liczenie PrimeArray jak wczesniej

        int blockSize = 65536; // 64 KB - optymalne dla cache L1/L2
        int numberOfBlocks = (n - m) / blockSize + ((n - m) % blockSize != 0);

        // Glowna petla rownolegla. Schedule(dynamic) najlepiej rozklada obciazenie w
           sicie segmentowym.
        #pragma omp parallel for schedule(static)
        for (int i = 0; i < numberOfBlocks; i++) {
                int low = m + i * blockSize;
                int high = low + blockSize - 1;
                if (high > n) high = n;

                // Rownolegla inicjalizacja fragmentu tablicy - dane trafiaja prosto do
                   cache rdzenia
                memset(&result[low - m], true, (high - low + 1) * sizeof(bool));

                for (int j = 2; j * j <= high; j++) {
                        if (primeArray[j]) {
                                int firstMultiple = (low / j) * j;
                                if (firstMultiple < low) firstMultiple += j;
                                if (firstMultiple == j) firstMultiple += j;

                                for (int k = firstMultiple; k <= high; k += j) {
                                        result[k - m] = false;
                                }
                        }
                }
        }

        //zliczanie prime_counter i zwalnianie pamieci jak wczesniej
}
```

Listing 7: Kod programu k5.cpp

3 Prezentacja wyników i omówienie przebiegu eksperymentu.

Zasady pracy z oprogramowaniem VTune.

Porównanie podstawowych parametrów wywołania dla kodów k1-k5.

Porównania zostały wykonane dla kodów przedstawionych w poprzedniej sekcji, z tymi samymi schedulingami i rozmiarami bloków na zakresie liczb od 1 do 100 milionów.

	k1	**k2**	**k3**	**k3a**	**k4**	**k4a**	**k5**
Elapsed Time [s]	23,750		0.660	0.130	0.381		
Prędkość przetwarzania [mln l. pierwszych/s]	4.210		14.394	35.714	15.121		
Instructions retired [mln]	405,744		0,515	0,777	2,179		
Clockticks [bilionów tików]	105,112		2,914	0,566	19,808		
Effective PCU	7.7%		4.7%	7.1%	42.4%		
Badania kodów równoległych	**k1**	**k2**	**k3**	**k3a**	**k4**	**k4a**	**k5**
Przyspieszenie względem k3a	-		-	-	0.34%		
Efektywność [Acc/Cores]	-		-	-	0.028		

Tablica 2: Porównanie podstawowych parametrów wywołania dla kodów k1-k5.

Performance Cores	**k1**	**k2**	**k3**	**k3a**	**k4**	**k4a**	**k5**
Retiring	66.0%		1.2%	0.0%	1.1%		
Front End Bound	11.4%		0.0%	3.1%	3.6%		
Back End Bound	3.3%		100.0%	0.0%	78.7%		
Memory Bound	0.2%		100.0%	0.0%	67.4%		
Core Bound	3.1%		0.0%	0.0%	11.4%		
Efficient Cores	**k1**	**k2**	**k3**	**k3a**	**k4**	**k4a**	**k5**
Retiring	0.0%		0.0%	0.0%	3.0%		
Front End Bound	0.0%		0.0%	0.0%	1.3%		
Back End Bound	4.1%		0.0%	0.0%	90.8%		
Memory Bound	0.0%		0.0%	0.0%	53.0%		
Core Bound	4.1%		0.0%	0.0%	37.8%		

Tablica 3: Wyniki analizy przyczyn ograniczeń wydajności dla kodów k1-k5 z podziałem na Performance i Efficient Cores.

Porównanie działań kodów dla różnych zakresów liczb.

W ramach tego eksperymentu zmierzyliśmy Ellapsed Time dla kodów k1-k5 dla trzech różnych zakresów liczb: <2; max>, <max/2; max> oraz <2; max/2>, gdzie `max` to 10^8.

	k1	**k2**	**k3**	**k3a**	**k4**	**k4a**	**k5**
<2; max>							
<max/2; max>							
<2; max/2>							

Tablica 4: Porównanie prędkości działania dla różnych zakresów liczb w kodach k1-k5.

Badanie wpływu różnych szeregowań dla czasu przetwarzania kodów k2, k4, k4a i k5.

Porównanie parametrów przetwarzania kodów k3 i k3a.

Wyniki eksperymentu wyznaczania wielkości bloku dla kodów k3a i k5.

Eksperyment - zwiększenie zakresu działania kodu k5 do 1 miliarda.

W czasie przeprowadzania eksperymentu k5 zauważyliśmy w teście threading niespodziewanie niskie CPU utilization, co skłoniło nas do przeprowadzenia kolejnego eksperymentu. Postanowiliśmy przeprowadzić dodatkowe badanie tym razem dla zakresu liczby zwiększonego o 1 potęgę 10 (więcej już by się nie dało ze względu na ograniczenia pamięciowe). Poniżej zamieściliśmy wykresy z porównaniem histogramów CPU utilization dla zakres 1exp8 i 1exp9.

./img/k5_108.png

Rysunek 1: CPU utilization dla kodu 5 dla zakresu 1exp8.

Dzięki temu porównaniu widzimy, że kod nie działa źle, czas w stanie 0/1 jest stały i jest to czas spędzony w sekcji sekwencyjnej (liczenie małego sita). Jest to dla obu uruchomień około 0.3 sekundy. Zmienia się między nimi zaś czas spędzony w sekcji równoległej (0.158 dla 1exp8 i 0.404 dla 1exp9). Wraz ze zwiększeniem się tego czasu możemy też zaobserwować wzort czasu w którym pracuje wiele wątków (co oznacza, że dobrze dzielą się zadaniami, mimo zwiększenia czasu w części równoległej nie zwiększyła się sumaryczna liczba czasu w stanie 0/1)

./img/k5_109.png

Rysunek 2: CPU utilization dla kodu 5 dla zakresu 1exp9.

4 Wnioski.